EXPERIMENT 1

Nivass C R - 241801193

Design and implement a three-layer neural network from scratch using Python. Train the network using the backpropagation algorithm with appropriate activation and loss functions. Apply the model to recognize handwriflen digits.

```
import numpy as np
from tensorflow.keras.datasets import mnist

(x_train, y_train), (x_test, y_test) = mnist.load_data()

x_train = x_train.reshape(-1, 784) / 255.0
x_test = x_test.reshape(-1, 784) / 255.0

x_train = x_train[:10000]
y_train = y_train[:10000]

def one_hot(y, classes=10):
    result = np.zeros((len(y), classes))
    result[np.arange(len(y)), y] = 1
    return result

y_train = one_hot(y_train)

def relu(x):
    return np.maximum(0, x)

def relu_derivative(x):
    return (x > 0).astype(float)

def softmax(x):
    x = x - np.max(x, axis=1, keepdims=True)
    e = np.exp(x)
    return e / np.sum(e, axis=1, keepdims=True)

np.random.seed(42)

W1 = np.random.randn(784, 128) * 0.01
b1 = np.zeros((1, 128))
W2 = np.random.randn(128, 10) * 0.01
b2 = np.zeros((1, 10))

learning_rate = 0.1
epochs = 10

for epoch in range(epochs):
    Z1 = x_train @ W1 + b1
```

```
    A1 = relu(Z1)

    Z2 = A1 @ W2 + b2
    A2 = softmax(Z2)

    loss = -np.mean(np.sum(y_train * np.log(A2 + 1e-8), axis=1))

    m = len(x_train)

    dZ2 = A2 - y_train
    dW2 = (A1.T @ dZ2) / m
    db2 = np.sum(dZ2, axis=0, keepdims=True) / m

    dA1 = dZ2 @ W2.T
    dZ1 = dA1 * relu_derivative(Z1)

    dW1 = (x_train.T @ dZ1) / m
    db1 = np.sum(dZ1, axis=0, keepdims=True) / m

    W1 -= learning_rate * dW1
    b1 -= learning_rate * db1
    W2 -= learning_rate * dW2
    b2 -= learning_rate * db2

    print("Epoch:", epoch + 1, "Loss:", loss)

Z1 = x_test @ W1 + b1
A1 = relu(Z1)
Z2 = A1 @ W2 + b2
A2 = softmax(Z2)

predictions = np.argmax(A2, axis=1)
accuracy = np.mean(predictions == y_test)

print("Test Accuracy:", accuracy)
```

```
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/mnist.npz
11490434/11490434 ━━━━━━━━━━━━━━━━━━━━ 1s 0us/step
Epoch: 1 Loss: 2.3031298028202487
Epoch: 2 Loss: 2.3019687373177407
Epoch: 3 Loss: 2.3008097034344406
Epoch: 4 Loss: 2.299640455040549
Epoch: 5 Loss: 2.2984499649084005
Epoch: 6 Loss: 2.2972269078546974
Epoch: 7 Loss: 2.295959470797304
Epoch: 8 Loss: 2.2946354251355805
Epoch: 9 Loss: 2.2932401964528317
Epoch: 10 Loss: 2.2917608617392626
Test Accuracy: 0.4025
```